Lumen Row Project Documentation

A human reviewed data quality agent built with Python, LangChain and LangGraph

Author: Krishna Kallamadi | Prepared 25 October 2026

The Primer: Your one-liner

Lumen Row helps data operations analysts turn customer tables into validated CSV imports in a local review workspace, replacing manual checking and spreadsheet correction; it uses four read-only tools autonomously, retains original files, rules, plans and approval checkpoints locally until deletion, and requires human approval before staging repairs and exporting, with a target of completing at least 9 of 10 known test scenarios within five minutes each.

The target measures the complete journey to a checked output. The current ten-case evaluation uses fictional data and scripted human decisions; it does not measure real user deliberation or live-model accuracy.

Project overview

Customer imports often fail because dates use different formats, countries use inconsistent labels, identifiers are missing, or rows are duplicated. Lumen Row makes those problems visible, proposes traceable repairs, and asks the user for facts it cannot establish. A valid file must satisfy the declared rules and pass both approval gates before export.

Scope

This release accepts CSV, TSV, XLSX, flat JSON, JSONL and NDJSON, then normalizes one table for customer-import checks. It supports required fields, uniqueness, exact duplicates, declared date formats, category aliases and export-safety checks. Output is a new CSV with an audit JSON and validation summary.

The app runs for a trusted local user. Authentication, multi-user tenancy, fuzzy entity merging, nested JSON, OCR, PDF extraction and preserving Excel formatting are outside the current scope. It cannot establish a missing customer fact merely because a replacement would pass validation.

GitHub repository: https://github.com/krish339-ai/DataQualityAgent

Author on LinkedIn: https://www.linkedin.com/in/krishna-kishore-kallamadi-a22b498/

The Framework

The framework connects the user's task to the agent's actions, memory, boundaries and measurable outcome.

Framework question	Implementation
Agent goal	Turn a customer table into a validated import file with explicit repairs and a readable audit.
Where do people use it	A local Streamlit application with a product landing page, review hub, guided upload and saved reviews.
What steps does it take in order	Import and preview; profile and validate; investigate; clarify and preview repairs; approve repairs; stage and revalidate; approve export; deliver CSV and audit.
What can it actually do	READ: profile_dataset, lookup_rules, validate_dataset and preview_repairs. WRITE: Engine.apply stages an approved copy; Engine.export produces approved deliverables. The two writes are guarded workflow actions, not model tools.
What does it need to remember	SQLite saves workflow state and approval pauses; job files preserve source, rules, plans, staged data and audit until deletion. Unsaved edits stay in the current app session; submitting saves the proposal durably.
What should it never do	Invent missing IDs or country facts, execute model-supplied code, overwrite the original, skip approval, export with blockers or expose unrelated customer fields to the model.
Human in the loop	A user supplies unknown facts and reviews the exact plan before staging. A second approval authorizes export; optional value suggestions need a payload preview and fresh consent.
What happens when something breaks	Bounded retries and decision budgets stop runaway model calls. Local findings remain visible and eligible investigation failures offer manual correction; invalid or stale approvals prevent writes.
How do you know it worked	The final output matches expected records, passes every blocking check and has an audit. The target is at least 9 of 10 known end-to-end cases completed within five minutes each, with scripted decisions in the automated harness.

Architecture and implementation

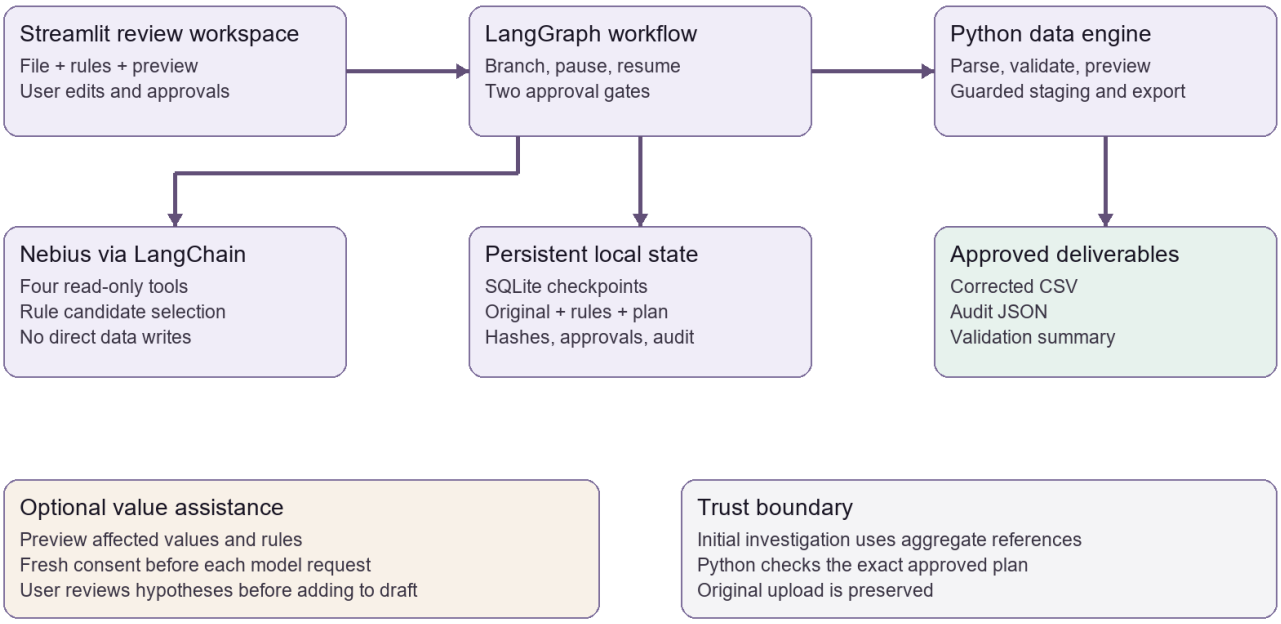


Figure 1. The model investigates; the workflow controls state and approvals; Python validates and writes.

LangGraph is the coordinator. Its nodes are start, investigate, preview, clarify, approve_repairs, apply, revalidate, approve_export and export. Conditional branches route unresolved issues back to clarification. Interrupts pause execution for a user decision, and SQLite checkpoints allow that same job to resume.

Component	Responsibility
app.py	Upload wizard, named reviews, correction editors, approvals and downloads.
workflow.py and engine.py	Durable transitions, rule checks, source preservation and guarded writes.
reasoning.py	LangChain tools, Nebius or Fireworks clients, bounded model investigation.
imports.py and schemas.py	Six input extensions, parser limits, normalized tables and rule contracts.
assistance.py and value_suggestions.py	Rule-backed drafts, consented hypotheses and local validation of suggestions.
telemetry.py	Optional aggregate LangSmith summaries; full customer-state tracing is disabled.

A single tool-using investigator is sufficient for this scope. The design does not require a team of autonomous agents, web search, vector retrieval or Mem0. The persistent business workflow is the key agentic behavior.

Approvals memory and failure recovery

A proposal is an intended change, not permission to write. The user can revise a draft and inspect its before-and-after result before saving it for review. Repair approval binds to the job, original source, rules and exact plan; changing any of them invalidates the old permission.

Python creates a separate staged copy after repair approval and validates it again. Export approval additionally checks the staged bytes. A failed validation or stale approval stops the write path. The original file remains available throughout.

Event	User-visible response
Impossible date or unknown country	Show the blocking finding and ask for a known correction. A valid-looking hypothesis is still unverified.
Model returns an invalid plan twice	Stop the investigation without changing data; show local findings and offer Continue with manual corrections when recovery is eligible.
Provider timeout or rate limit	Retry transient errors up to three attempts. If exhausted, explain the failure and keep the saved job available.
Data or plan changes after approval	Reject the stale approval and require review of the new content.
Reset a job	Create a fresh run from the original input and parsing choices. Keep the earlier run and require new approvals.
App restarts	Resume submitted plans and workflow checkpoints. Unsaved browser-session drafts are not promised across a restart.

Privacy and operating limits

Initial investigation sends aggregate counts and opaque candidate references to the model. Optional corrected-value assistance shares affected field names, current values, issue types and relevant rules only after the user previews and authorizes that request. Unrelated columns and unsaved edits are excluded. Requests are limited to 20 affected cells and 32 KiB.

Files and saved jobs stay under the local data directory until explicitly deleted. The model has no shell or write tool. Investigation has a 12-decision budget and at most four tool calls per decision. Imports are bounded to 10 MiB, 100,000 rows, 200 columns and one million cells, with additional parser expansion limits.

Datasets and validation rules

All committed examples are fictional and use example.com addresses. No real customer jobs, API credentials, local databases or generated user outputs are part of the repository.

Dataset	Purpose and expected result
samples/customers_positive.csv	Eight input records with duplicates, date formats and country aliases. Rule-backed repairs produce seven records and no blocking issues.
samples/customers_negative.csv	Two records: an impossible February 30 date, unknown ZZ country and missing customer ID. Three blockers require human knowledge.
samples/formats/	Equivalent positive fixtures in TSV, XLSX, JSON, JSONL and NDJSON for parser and workflow checks.
samples/practice/	Six new records with seven initial issues, four rule-backed changes and three facts to supply. Known fixture answers are included separately.
dq_agent/__main__.py fixtures	Ten small cases with exact expected outputs, covering clean data, duplicates, aliases, dates, missing ID and combinations.

How the example rules work

Rule	Meaning and example
Required fields	customer_id, name, email, signup_date and country must exist and be nonempty in the interactive examples.
Unique identifier	customer_id must be unique. Treat it as text so 00104 keeps its leading zeros.
Date conversion	Input %m/%d/%Y means MM/DD/YYYY. Output %Y-%m-%d means YYYY-MM-DD. 10/05/2026 becomes 2026-10-05.
Country categories	US, CA and GB are allowed. Explicit aliases such as Canada -> CA and uk -> GB can be repaired; ZZ has no known mapping.
Limits of these rules	Required email does not mean email syntax or ownership is checked. A passing date format does not prove the intended signup date.

Worked negative scenario

Create a Negative test job from the review hub. Record 1 contains 02/30/2026 and ZZ; record 2 has no customer_id. The expected initial count is three blockers. These are data problems even if the model investigation also fails.

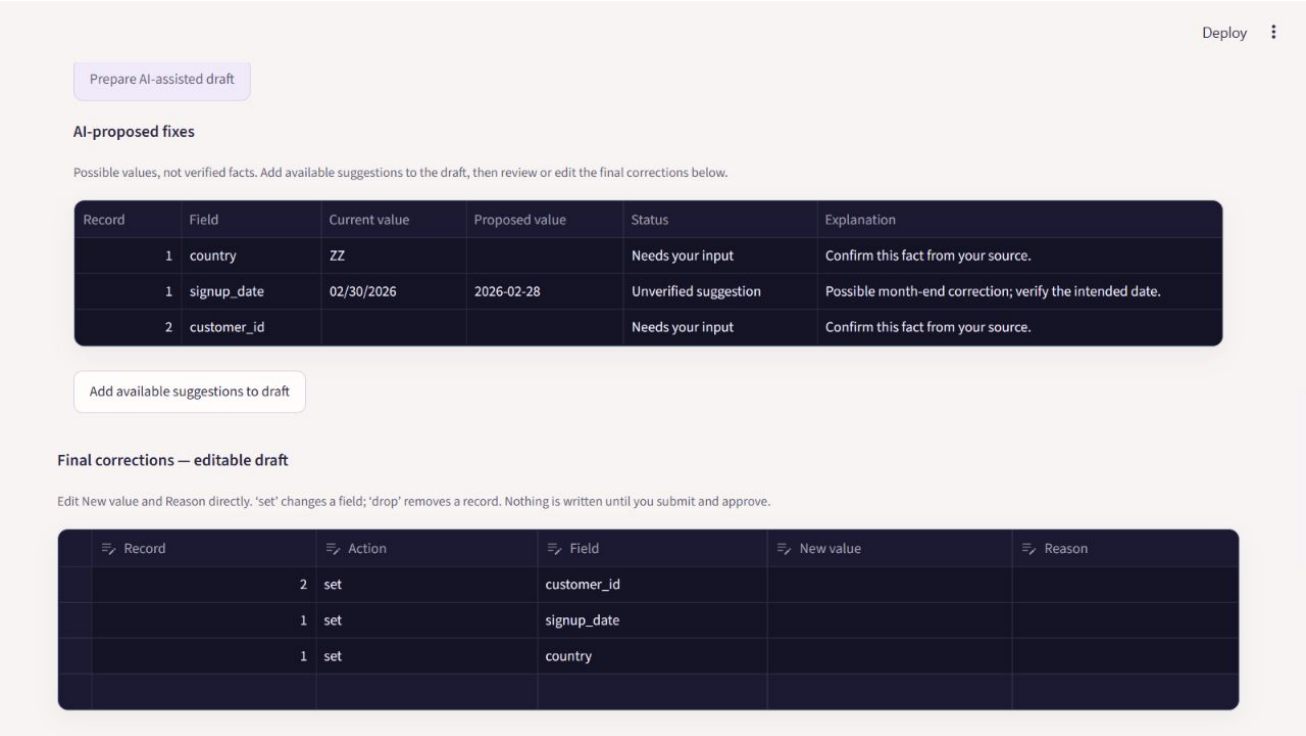


Figure 2. Compact suggestion and correction tables with fictional data. This UI capture uses controlled suggestions; live model wording may differ.

Prepare AI-assisted draft includes all affected fields. In live mode, inspect the payload preview and authorize the request first. Add available suggestions to draft, then review or edit the final correction table. Country and identifier values remain blank when there is no evidence; a suggested month-end date still needs confirmation.

Record	Field	Known sample correction	Reason
1	signup_date	2026-02-28	Intended date supplied for this fictional test.
1	country	CA	Known sample country; not inferred from ZZ.
2	customer_id	102	Known unique sample identifier.

These are fixture answers, not facts to reuse on real records. Enter them with reasons, confirm zero remaining blockers, and submit the revised plan. Approve the exact repairs, inspect the staged validation result, then approve export. Expected output: two corrected records, zero blocking issues and an audit.

Coding prompts and development iterations

Codex was used as an AI coding assistant for design, Python scaffolding, interface changes, debugging, documentation and tests. The build was steered through repeated user review of the actual interface and workflow. The excerpts below are shortened from those requests.

Prompt excerpt	Implementation outcome
Define the scope and architecture using the handout's agent framework ... Python, LangChain, and LangGraph.	A primer and framework were reviewed before implementing the stateful workflow.
I should see the correct issue count and be able to enter corrections even if the model fails.	Local validation drives the count; eligible failed investigations allow manual recovery in the same job.
While editing date field ... use calendar ... country ... drop down ... acceptable values per rules.	Optional field controls use date pickers and rule-derived categories; text fields preserve identifiers.
By default all should be available in the same tabular format ... AI Proposed fixed table just above final table.	All affected fields are prepared together, with compact suggestions above the editable draft and a batch-add action.

Iterations that changed the product

The first version handled CSV and paused on unresolved facts. Live-provider configuration and error handling were then added. Negative testing exposed a misleading zero issue count and a blocked manual-edit path after model failure; both were corrected and covered by regression tests.

The interface evolved from a plain form into a themed landing page and review hub. Readable job names, reset-as-new-run behavior, saved-review search and guided File, Rules and Preview steps made the workflow easier to navigate. Parser expansion added five more extensions while retaining a single validation and approval engine.

Value assistance initially required too many separate steps. It now prepares the draft and possible replacements from one action, while retaining a payload preview and consent. Field-level forms became optional and collapsed, and suggestions moved into a compact table.

The generated landing artwork and matching light-mode edit prompts are recorded in assets/hero-prompts.txt and assets/light-matching-prompt.txt. The scene and animation illustrate records being reviewed; they are not a live view of processing.

Verification and observations

Release verification on 25 October 2026: 422 automated tests passed, with 0 failures and 0 errors. The ten-case offline evaluation completed 10 of 10 scenarios with exact expected outputs; the slowest case took 1.047 seconds on the local test machine.

Evidence	What it establishes	What it does not establish
Regression suite	Rules, parsers, approvals, persistence, recovery, drafts, consent and UI behavior under test.	Production certification or zero defects.
10 of 10 offline cases	Known end-to-end fixtures reach exact expected CSV outputs through scripted approval decisions.	Live-model accuracy, real user completion time or performance on arbitrary uploads.
Provider integration	Nebius is implemented through LangChain; provider responses and failures have controlled test coverage.	Every model in the provider catalog will behave identically or remain available.

Reproduce the checks after setup: python -m pytest; python -m dq_agent evaluate --root .eval-data. The committed docs/test-results.json and docs/evaluation-results.json contain aggregate evidence without saved customer jobs or machine-specific paths.

Learnings and observations

Validation and factual truth are different. Python can prove that CA is allowed, but cannot determine whether an unknown customer lives in Canada. The interface must distinguish deterministic transformations, unverified AI hypotheses and facts supplied by the user.

Human approval must bind to exact content. A generic checkbox is insufficient if the source, rules, plan or staged bytes can change. Checkpoints and content hashes make resumed work reviewable and protect the write boundary.

Model failure should not erase deterministic evidence or trap the user. Keeping local validation independent makes the workflow useful when a provider times out, produces malformed output or exhausts its budget.

Interface verification matters alongside unit tests. Repeated review caught unclear date-entry expectations, oversized controls and excessive correction steps. Compact tables and rule-aware controls reduced friction while keeping explicit approvals.

Next improvements include broader evaluation with independently labeled datasets, measured live-model reliability and cost, richer export formats, and access controls before any multi-user deployment.

Setup and demonstration guide

Clone the repository and use Python 3.11 or newer. On Windows, run scripts/bootstrap.ps1 and scripts/start.ps1. Open http://127.0.0.1:8501. Bootstrap creates a virtual environment and copies the safe .env.example template only when .env does not already exist.

Use Offline demonstration for reproducible local checks. For a live run, configure NEBIUS_API_KEY and NEBIUS_MODEL in .env, set DQ_PROVIDER=nebius, restart the server, and choose Nebius Token Factory for new jobs. Never show the key while recording. Existing jobs retain their original provider mode.

Loom walkthrough within five minutes

Time	Show and explain
0:00 to 0:30	Introduce Lumen Row, the customer-import problem, target user and Python/LangChain/LangGraph stack.
0:30 to 1:10	Open the workspace. Show file choices, rules and local preview, then create a negative sample job.
1:10 to 2:20	Explain the three blockers. Show the payload preview and AI-assisted draft if using Nebius; distinguish unknown facts from possible corrections.
2:20 to 3:20	Supply known fixture answers, inspect the before/after preview, submit the plan and explicitly approve the repair.
3:20 to 4:05	Show zero blockers on the staged copy, approve export and open the corrected CSV plus audit. Briefly show a pre-created positive result.
4:05 to 4:45	Show the architecture diagram and explain saved checkpoints, failure recovery, exact-plan approvals and how Codex helped build and iterate.
4:45 to 5:00	Show the repository and report links. State the test results with their offline/scripted limitations.

Prepare sample jobs and confirm provider access before recording. If a live request takes too long or fails, explain the visible failure and use the manual recovery path; do not present an offline run as a live-model response. Keep the final recording at five minutes or less.

Try the additional practice file

Upload samples/practice/customer-review-practice.csv with customer-review-rules.json. Expect seven initial issues and three unresolved facts after four rule-backed changes. Use expected-corrections.json only after trying the exercise; the complete plan should leave six output rows and no blockers.

Submission links and completion checklist

GitHub codebase: https://github.com/krish339-ai/DataQualityAgent

Google Docs project report link: add the shared link after importing this document into Google Drive.

Loom demonstration link: add the recording link after completing the walkthrough.

Submission form: https://forms.gle/waBYFkkf5xxmcouV9

Upload this DOCX to Google Drive and open it with Google Docs. Check that tables and figures remain readable, set the document's sharing access for the intended reviewers, and copy its share link. Add the Loom link and repository URL to the submission form after verifying access.

Requested content	Where to find it
Project overview and end-to-end task	The Primer, Project overview and Scope.
Agent framework and architecture	The Framework, architecture diagram and approval/memory explanation.
Datasets used and examples	Datasets and validation rules; worked negative scenario; practice fixtures.
Prompts used during AI-assisted coding	Coding prompts and development iterations; artwork prompt files.
Iterations and learnings	Development iterations, verification and observations.
Codebase and demo	Public GitHub link above; five-minute Loom outline; add the final recording link.

About the author

I'm Krishna Kallamadi, a data enthusiast with 16+ years of experience with various on-prem and cloud databases across healthcare and insurance. I began my career in 2009 and have spent more than a decade supporting mission-critical healthcare databases. My work spans database administration, performance tuning, data analytics, data migration, data engineering, data architecture, cloud engineering, and AI engineering, with a focus on automation and data security.

I've expanded into AI, earning the AWS Certified AI Practitioner certification and building AI-powered database intelligence tools. I'm working toward a future as an AI Data Architect, bringing together database expertise, data engineering, and a security-first approach to intelligent, autonomous platforms.

Connect on LinkedIn: https://www.linkedin.com/in/krishna-kishore-kallamadi-a22b498/